

# Adjoint, Eligibility Traces and Temporal Credit

Backpropagation through time, forward-mode traces, and why a decaying memory is its own backward pass

## Why this tutorial

A recurrent model's state carries information forward in time; learning must carry *credit* for later losses back to the earlier computations that caused them. There are two exact ways to do this: run the computation backwards (the adjoint, as in backpropagation through time) or carry sensitivities forwards (eligibility traces). For the linear decaying memories in our models both are unusually cheap and have a physical reading. This tutorial derives both, proves they agree, and quantifies what truncation costs.

## 1 Setting

State  $s_t \in \mathbb{R}^n$  evolves by  $s_t = F(s_{t-1}, x_t; \theta)$  from inputs  $x_t$ ; the loss is  $L = \sum_{t=1}^T \ell_t(s_t)$ . Write the local Jacobians  $J_t = \partial F / \partial s_{t-1}$  (an  $n \times n$  matrix),  $e_t = \partial F / \partial \theta$  (an  $n \times P$  matrix, the *local* parameter sensitivity at step  $t$ ) and  $c_t = \partial \ell_t / \partial s_t$  (the local cotangent).

## 2 Reverse mode: the adjoint

**Definition 1** (Adjoint).  $\lambda_t = \frac{dL}{ds_t}$ , the total effect of  $s_t$  on the loss, through every later step.

**Proposition 2.**

$$\lambda_T = c_T, \quad \lambda_t = c_t + J_{t+1}^\top \lambda_{t+1}, \quad \nabla_\theta L = \sum_{t=1}^T e_t^\top \lambda_t.$$

*Proof.*  $s_t$  affects  $L$  directly through  $\ell_t$  and indirectly through  $s_{t+1} = F(s_t, \dots)$ ; the chain rule gives the recursion.  $\theta$  enters every step; each entry contributes  $e_t^\top$  times the total effect of  $s_t$ .  $\square$

This is backpropagation through time (BPTT). It needs all states  $s_1, \dots, s_T$  stored (memory  $O(nT)$ ) and a backward sweep that starts only after the sequence ends.

## 3 Forward mode: eligibility traces

**Definition 3** (Eligibility trace).  $S_t = \frac{ds_t}{d\theta}$  ( $n \times P$ ), the total sensitivity of the current state to the parameters.

**Proposition 4.**

$$S_0 = 0, \quad S_t = J_t S_{t-1} + e_t, \quad \nabla_\theta L = \sum_{t=1}^T S_t^\top c_t.$$

*Proof.* Differentiate  $s_t = F(s_{t-1}, x_t; \theta)$ : the direct term  $e_t$  plus the path through  $s_{t-1}$ . Then  $d\ell_t / d\theta = c_t^\top S_t$ .  $\square$

This is real-time recurrent learning (RTRL). No history is stored and each loss can be applied the moment it arrives. The price is carrying  $S_t$ :  $n \times P$  numbers, updated with an  $n \times n$  by  $n \times P$  product per step, generally far more expensive than BPTT.

**Proposition 5** (The two agree).  $\sum_t e_t^\top \lambda_t = \sum_t S_t^\top c_t$ .

*Proof.* Unroll:  $\lambda_t = \sum_{u \geq t} (J_{t+1}^\top \cdots J_u^\top) c_u$  and  $S_u = \sum_{t \leq u} (J_u \cdots J_{t+1}) e_t$ . Both sides equal  $\sum_{t \leq u} e_t^\top (J_{t+1}^\top \cdots J_u^\top) c_u$ : the same double sum, summed in the other order.  $\square$

## 4 Diagonal decaying memories: both directions become cheap

Our temporal memories are banks of independent modes. Mode  $m$  evolves as

$$z_{t,m} = a_{t,m} z_{t-1,m} + b_{t,m}, \quad a_{t,m} = e^{(-r_m + i\omega_m)\Delta t_t},$$

a complex number that decays at rate  $r_m$  and rotates at frequency  $\omega_m$  over the elapsed time  $\Delta t_t$ ;  $b_t$  is the write. The Jacobian is diagonal, so:

- **Traces stay small.** A parameter that affects only mode  $m$  needs only that mode's trace:  $S_t = a_t S_{t-1} + \partial b_t / \partial \theta$ . Trace storage equals parameter count, not  $n \times P$ .
- **The adjoint is the memory run backwards.** For a real loss of complex states (use the real 2-vector  $(\Re z, \Im z)$ ), equivalently the complex gradient  $\partial L / \partial \Re z + i \partial L / \partial \Im z$ , multiplication by  $a$  is a rotation-and-scaling whose transpose is multiplication by  $\bar{a}$ :

$$\lambda_{t,m} = c_{t,m} + \overline{a_{t+1,m}} \lambda_{t+1,m}.$$

Same decay, opposite rotation, run in reverse time: a delay line carries the echo back with the same attenuation.

### Key idea

For a decaying, rotating memory the backward pass is not a second machine; it is the forward machine time-reversed. And the forward-mode trace is exact and history-free at the cost of one accumulator per parameter. Per-event learning (update the weights at every event, no stored sequence) is therefore exact for these memories at fixed weights.

## 5 Truncation and the credit horizon

Unrolling the adjoint for one mode,  $\lambda_t = \sum_{j \geq 0} (\prod_{k=1}^j \overline{a_{t+k}}) c_{t+j}$ . If  $|a| \leq \rho < 1$  and  $|c| \leq C$ , keeping only  $j \leq H$  errs by at most

$$C \sum_{j > H} \rho^j = C \frac{\rho^{H+1}}{1 - \rho}.$$

Fast modes ( $\rho$  small) need a short credit horizon; slow modes need a long one. With  $\rho = e^{-r\Delta}$ , a relative error  $\varepsilon$  needs about  $\ln(1/\varepsilon)/(r\Delta)$  steps. *Truncating the trace* (keeping only the current step's local term, as in e-prop) is the extreme case  $H = 0$  applied to the cross-step path.

## 6 Depth

Stack two memories: layer 2 reads layer 1. A layer-1 parameter now reaches layer-2 state through every past layer-2 input, so the exact layer-2 trace with respect to layer-1 parameters has  $n_2 \times P_1$  entries. Exact per-event learning across depth costs storage proportional to (upper state)  $\times$  (lower parameters), not to sequence length. BPTT's cost is the reverse. Which is cheaper depends on depth, width and sequence length.

## Exercises

**Exercise 1.** Scalar memory  $s_t = as_{t-1} + wx_t$ , loss  $L = \sum_t \frac{1}{2}(s_t - y_t)^2$ . Write  $\lambda_t$  and  $S_t = ds_t/dw$  and check  $\sum_t x_t \lambda_t = \sum_t S_t(s_t - y_t)$  for  $T = 2$ .

*Solution.*  $c_t = s_t - y_t$ ,  $\lambda_2 = c_2$ ,  $\lambda_1 = c_1 + ac_2$ ;  $e_t = x_t$ ;  $S_1 = x_1$ ,  $S_2 = ax_1 + x_2$ . Left:  $x_1(c_1 + ac_2) + x_2c_2$ . Right:  $x_1c_1 + (ax_1 + x_2)c_2$ . Equal.

**Exercise 2.** Why must the backward pass use  $\bar{a}$  and not  $a$  for a complex mode?

*Solution.* As a real  $2 \times 2$  map, multiplication by  $a = \alpha + i\beta$  is  $\begin{pmatrix} \alpha & -\beta \\ \beta & \alpha \end{pmatrix}$ ; its transpose is multiplication by  $\bar{a}$ .

**Exercise 3.** A mode has  $r = 0.05$  per unit time and events arrive every  $\Delta = 1$ . How many steps of exact credit are needed for 1% truncation error (relative to the bound's scale)?

*Solution.*  $\rho = e^{-0.05} \approx 0.951$ ; need  $\rho^{H+1} \leq 0.01(1 - \rho)$ , i.e.  $H + 1 \geq \ln(0.01 \cdot 0.049) / \ln 0.951 \approx 152$ .

### Where this appears in our work

Theory note 160 §1 (adjoint = time-reversed memory, checked to  $10^{-16}$ ), §7 (per-event trace learning beats batched BPTT per pass on Taxi), §12 (exact traces across depth); `experiments/credit/online_race.py`, `online_deep.py`; note 164 (the adjoint as a Bellman equation, Tutorial 5) and note 165 §8 (which credit method for which mode).